

ERTS: Assignment 2

Simon Møller, student number: 202204535
Mark Myrtoft Bøgelund, student number: 202206634

Date: September 25, 2026

Introduction

The purpose of this assignment is to learn the design flow for a system on a programmable chip. Using Vivado, SDK, and HLS, we explore the relationship between the Zynq Processing System (PS) and Programmable Logic (PL), interface drivers, command-line control, custom IP, and hardware acceleration.

We began by following the setup instructions to configure the ZYBO board, connect it to the development computer, and prepare the required Xilinx design tools. This was initially quite challenging, since our pc runs on Windows 11, and the old software was not fully compatible. Eventually, we got it working. This provided the hardware and software platform for the exercises that follow.

Exercise 3: Create a Console Application

Objective

In this exercise, we create a small console application for the ZYBO board. It receives commands over USB-UART and uses them to control the LEDs through the switches and timer.

Command interpreter

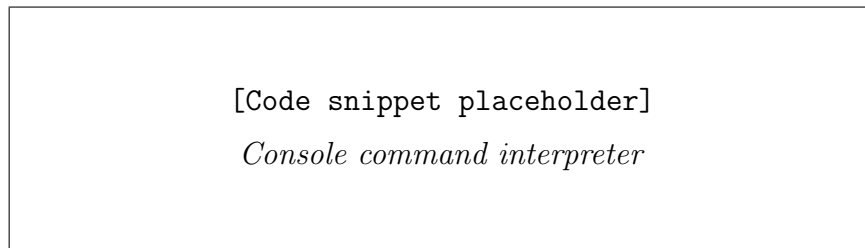


Figure 1: Console command interpreter

The application will read a command from the terminal and use a **switch** statement to select the requested operation.

Command 1: Switches to LEDs

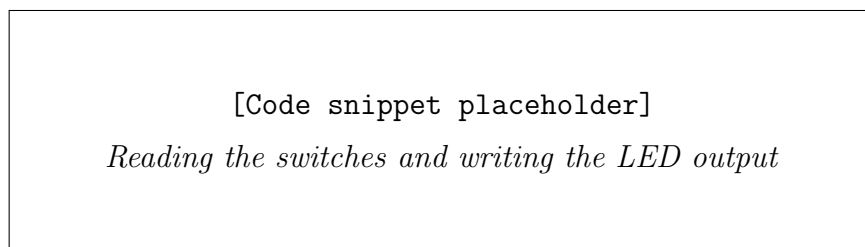


Figure 2: Reading the switches and writing the LED output

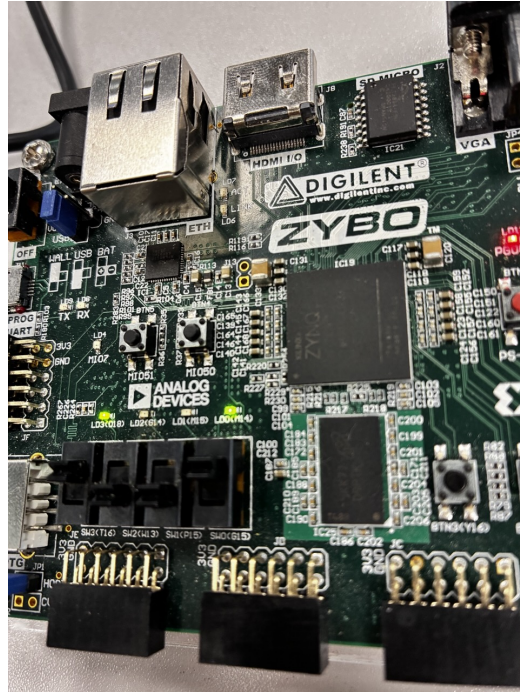


Figure 3: The LEDs reflect the binary state of the switches.

With command 1, the LEDs light up according to the switch positions.

Command 2: Timer-driven binary counter

[Code snippet placeholder]
XScuTimer initialization and one-second counter

Figure 4: XScuTimer initialization and one-second counter

Exercise 4: Create a Matrix Multiplication

Objective

In this exercise, we implement a software matrix multiplication on the ZYBO board. The program uses two 4x4 matrices, calculates the product, and reports the execution time through the USB-UART interface.

Matrix multiplication

The first matrix contains the values from 1 to 16. For the second matrix, the assignment gives B^T , so if:

$$B^T = \left(\begin{bmatrix} 1 & 1 & 1 & 1 \\ 2 & 2 & 2 & 2 \\ 3 & 3 & 3 & 3 \\ 4 & 4 & 4 & 4 \end{bmatrix} \right)^T = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 1 & 2 & 3 & 4 \\ 1 & 2 & 3 & 4 \\ 1 & 2 & 3 & 4 \end{bmatrix}.$$

Then:

$$B = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 2 & 2 & 2 & 2 \\ 3 & 3 & 3 & 3 \\ 4 & 4 & 4 & 4 \end{bmatrix}$$

We used the matrices A and B as defined above for the multiplication, which is performed using nested loops, where each element in the result is calculated as the sum of four products.

Following the operations shown in the figure below, each result element is calculated from one row of A and one column of B .

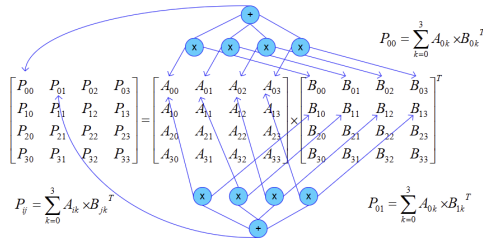


Figure 7: Matrix multiplication operations for calculating the elements of P .

We made the multiplication and transposition in the same loops. This was confusing at first, but we eventually got it and arrived at the correct result.

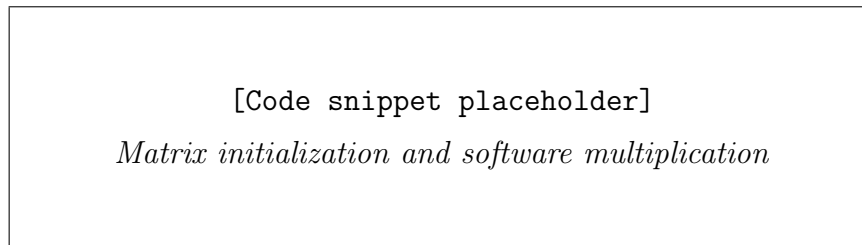


Figure 8: Matrix initialization and software multiplication

```

CMD :>
Matrix A:
1      2      3      4
5      6      7      8
9      10     11     12
13     14     15     16

Matrix B^T:
1      2      3      4
1      2      3      4
1      2      3      4
1      2      3      4

Matrix P:
10     20     30     40
26     52     78     104
42     84     126    168
58     116    174    232

Execution time: 1102 clock ticks
CMD :>

```

Figure 9: Matrix multiplication result and execution time.

The result matrix is displayed correctly, and the software implementation takes 1102 clock ticks in this run. This result will later be used for comparison with the hardware-accelerated version.

Exercise 5: Create a Hardware IP Acceleration

Exercise 6: HLS exercise

Exercise 7: HLS exercise implementing a Neural Network